

# Development Plan - Multi-Provider Object Storage SaaS

## Tujuan Produk

Membangun SaaS multi-tenant yang memungkinkan organisasi:

- Mendaftarkan beberapa S3 atau S3-compatible provider.
- Menghubungkan physical bucket yang sudah dimiliki.
- Menyajikannya sebagai logical namespace atau virtual storage.
- Mengelola object melalui file explorer.
- Memberi akses client berdasarkan prefix dan actions.
- Mengeluarkan satu API credential tanpa membuka provider credential.
- Menerapkan storage quota, request limit, dan bandwidth policy.
- Menentukan placement object berdasarkan priority, weight, health, dan threshold.
- Melihat usage dan audit log.

Produk harus menyembunyikan keputusan provider dari business application pelanggan, tetapi tidak harus menyembunyikan provider hostname pada presigned URL dalam MVP.

## Asumsi yang Telah Disepakati

| Area                     | Keputusan                                    |
|--------------------------|----------------------------------------------|
| Scale                    | Growth-ready MVP                             |
| Tenancy                  | Multi-organization atau multi-tenant         |
| Dashboard auth           | Short-lived JWT dan rotated refresh cookie   |
| Admin                    | Global platform admin dan organization admin |
| Repository               | npm workspaces monorepo                      |
| Frontend                 | Next.js App Router                           |
| API                      | Express dan TypeScript                       |
| Validation               | Zod                                          |
| Database                 | PostgreSQL dan Prisma                        |
| Gateway                  | Shared pool dengan opsi dedicated enterprise |
| Background jobs          | Worker dan Redis/BullMQ                      |
| Provider scope           | S3 dan S3-compatible                         |
| Initial client interface | Custom REST API                              |
| S3-compatible facade     | Fase lanjutan                                |

## Prinsip Scope

- Mulai dengan solusi paling sederhana yang memenuhi kebutuhan saat ini.
- Jangan membuat microservices sebelum ada kebutuhan scaling atau ownership yang nyata.
- Provider behavior dianggap berbeda sampai contract test membuktikan kompatibilitas.

- Authorization dan tenant isolation selalu ditegakkan di backend.
- Semua provider endpoint, API input, file metadata, dan external response dianggap untrusted.
- Destructive operation mendapat confirmation, audit, dan pembatasan khusus.
- File data tidak disimpan di PostgreSQL atau local disk gateway.

## Scope MVP

### Tenant dan identity

- Organization.
- User dan membership.
- Organization-scoped roles.
- Global platform role.
- Action-level permissions.
- Session management dan revocation.
- Append-only activity log.

### Provider dan bucket

- Provider presets.
- Generic S3-compatible endpoint.
- Encrypted provider credential.
- Connection test.
- Capability probing.
- Import existing bucket.
- Provider dan bucket health status.

### Virtual storage

- Logical namespace.
- Satu atau lebih placement target.
- Object index.
- Physical object location history untuk migration cutover dan retained source.
- File explorer.
- Prefix-based grants.
- API credential per client.
- Custom REST object API.

### Quota dan placement

- Storage quota.
- Reserved bytes.
- Maximum object size.
- Requests per second.

- Concurrent transfer limit.
- Priority placement.
- Weighted placement.
- Configured capacity threshold.
- Provider health eligibility.

## Operations

- Direct presigned upload.
- Optional proxied upload/download.
- Multipart upload.
- Cross-provider migration untuk current active object.
- Migration preflight, progress, pause/resume, retry, cancel sebelum cutover, dan delayed source cleanup.
- Idempotency.
- Orphan cleanup.
- Usage reconciliation.
- Structured logs, metrics, dan alerts.

## Di Luar MVP

- Complete S3 API compatibility.
- Replication dan cross-provider redundancy.
- Erasure coding.
- Non-S3 protocols.
- Cross-provider lifecycle normalization.
- Cross-provider versioning normalization.
- Private-network connector.
- Customer-managed encryption key.
- Automatic cost optimization berdasarkan live provider billing.
- Historical object-version migration.
- Automatic cost-based migration dan advanced bulk migration orchestration.
- Dedicated full stack per tenant.

## Arsitektur Delivery

```
app.example.com      -> Next.js web
api.example.com      -> Control API
storage.example.com  -> Shared gateway pool
tenant.storage.example.com -> Optional dedicated gateway
```

### Monorepo:

```
apps/web
apps/api
apps/worker
packages/shared
```

```
packages/db
packages/storage-core
packages/provider-s3
packages/auth
packages/observability
```

Lihat keputusan lengkap pada:

- Teknologi dan deployment: ./TECHNOLOGY.md
- Product feedback dan risiko: ./FEEDBACK.md

## Feature Modules

### 1. Authentication

Fitur:

- Login.
- Refresh token rotation.
- Logout current session.
- Logout all sessions.
- Session expiry handling.
- Password reset atau external auth provider pada fase berikutnya.

Rules:

- Access token short-lived.
- Refresh token berada dalam Secure, httpOnly cookie.
- Refresh token database value berbentuk hash.
- Cookie mutation dilindungi CSRF.
- Login, refresh, revoke, dan failed authentication diaudit.

### 2. Organization dan membership

Fitur:

- Organization CRUD oleh global admin.
- Organization profile oleh organization owner.
- Invite member.
- Disable member.
- Assign dan revoke role.
- Organization status: active, suspended, closed.

Rules:

- Suspended organization tidak dapat membuat upload session baru.
- Read/download behavior ketika suspended harus ditentukan oleh product policy.
- Platform support tidak otomatis mendapat akses file.

### 3. RBAC

Role awal:

| Role               | Scope        | Ringkasan                              |
|--------------------|--------------|----------------------------------------|
| Platform Admin     | Global       | Tenant, plans, platform operations     |
| Organization Owner | Organization | Semua konfigurasi organisasi           |
| Storage Admin      | Organization | Provider, bucket, namespace, object    |
| Developer          | Organization | API credential dan object sesuai grant |
| Auditor            | Organization | Usage dan audit read-only              |

### Permission keys:

```

organizations:create
organizations:read
organizations:update
organizations:suspend
members:create
members:read
members:update
members:delete
roles:create
roles:read
roles:update
roles:delete
roles:assign
permissions:manage
providers:create
providers:read
providers:update
providers:delete
providers:test
buckets:import
buckets:read
buckets:manage
namespaces:create
namespaces:read
namespaces:update
namespaces:delete
placement_policies:read
placement_policies:manage
folder_grants:read
folder_grants:manage
api_credentials:create
api_credentials:read
api_credentials:rotate
api_credentials:revoke
objects:list
objects:read
objects:write
objects:delete
migrations:create
migrations:read
migrations:control
usage:read
audit:read
platform:manage

```

### Baseline permission matrix:

| Resource/action              | Platform Admin        | Org Owner | Storage Admin | Developer | Auditor |
|------------------------------|-----------------------|-----------|---------------|-----------|---------|
| Organizations manage         | Yes                   | Own only  | No            | No        | No      |
| Members manage               | Support-limited       | Yes       | No            | No        | No      |
| Roles and permissions manage | Global templates      | Yes       | No            | No        | No      |
| Providers manage/test        | Metadata/support only | Yes       | Yes           | No        | Read    |
| Buckets import/manage        | No content access     | Yes       | Yes           | No        | Read    |
| Namespaces manage            | No content access     | Yes       | Yes           | Read      | Read    |
| Placement policy manage      | No                    | Yes       | Yes           | No        | Read    |

| Resource/action        | Platform Admin   | Org Owner      | Storage Admin  | Developer   | Auditor       |
|------------------------|------------------|----------------|----------------|-------------|---------------|
| Folder grants manage   | No               | Yes            | Yes            | No          | Read          |
| API credentials manage | No               | Yes            | Yes            | Scoped      | Read metadata |
| Objects list/read      | Break-glass only | Yes            | Yes            | By grant    | No by default |
| Objects write/delete   | No               | Yes            | Yes            | By grant    | No            |
| Migrations             | Read metadata    | Create/control | Create/control | No          | Read status   |
| Usage read             | Aggregate        | Yes            | Yes            | Scoped      | Yes           |
| Audit read             | Platform events  | Yes            | Yes            | Own actions | Yes           |

Every permission gate exists at:

- Backend endpoint.
- Service ownership check.
- Repository organization filter.
- UI route.
- UI action control.

#### 4. Provider connection

Provider form:

- Display name.
- Provider preset.
- Endpoint.
- Region.
- Access key ID.
- Secret access key.
- Optional session token.
- Addressing mode.
- TLS configuration.

Connection test:

1. Validate endpoint.
2. Apply SSRF controls.
3. Create provider adapter client.
4. HEAD or list permitted resource.
5. Probe selected capabilities.
6. Store sanitized result.
7. Record audit event.

Provider status:

PENDING  
HEALTHY  
DEGRADED  
UNHEALTHY  
DISABLED

Secret values never appear again after creation.

## 5. Bucket connection

MVP operations:

- List accessible buckets.
- Import bucket.
- Verify bucket access.
- Attach bucket to namespace.
- Disable connection.
- View capability and health.

Deferred:

- Delete physical bucket.
- Normalize lifecycle policy.
- Normalize bucket ACL.
- Normalize versioning.

## 6. Logical namespace

VirtualNamespace represents storage yang dilihat client.

Fields:

- Organization.
- Name dan slug.
- Status.
- Logical storage quota.
- Maximum object size.
- Default transfer mode.
- Placement policy.
- Versioning mode: disabled pada MVP.

Namespace memiliki placement targets menuju satu atau lebih BucketConnection.

## 7. File explorer

Features:

- List prefix dengan pagination.
- Search berdasarkan exact prefix atau indexed metadata.
- Breadcrumb.
- Upload.
- Download.
- Rename melalui copy-and-delete job.
- Delete.
- Object detail.

- Status badge untuk pending/failed objects.

Notes:

- Folder adalah prefix.
- Empty folder dapat direpresentasikan oleh zero-byte marker hanya jika diperlukan.
- Rename bukan operasi atomik pada S3.
- Large recursive delete atau rename dijalankan sebagai background job.

## 8. Folder grant

FolderGrant fields:

- Organization.
- Namespace.
- Prefix.
- Principal type: role atau API credential.
- Actions.
- Optional expiry.

MVP policy:

- Allow-only.
- Prefix normalized tanpa path traversal.
- Action list, read, write, dan delete.
- Credential hanya dapat mengakses satu namespace pada versi awal.
- List result tidak boleh membocorkan object di luar allowed prefix.

## 9. API credential

Fitur:

- Create.
- Show secret once.
- Rotate.
- Revoke.
- Expire.
- Assign prefix grants.
- Apply quota/rate profile.
- View last used timestamp.

Stored fields:

- Public key ID.
- Secret hash.
- Organization.
- Namespace.
- Status.



- Expiry.
- Last used.
- Created by.

## 10. Quota dan usage

Counters:

- usedBytes.
- reservedBytes.
- objectCount.
- activeUploadCount.
- requestCount.
- ingressBytes.
- egressBytes.

Quota evaluation:

```
available = quotaBytes - usedBytes - reservedBytes
```

Upload ditolak bila expected size melebihi available.

Rules:

- Reserve dalam database transaction.
- Idempotency key mencegah double reservation.
- Reservation mempunyai expiry.
- Successful completion memindahkan reservedBytes ke usedBytes.
- Abort atau expiry melepas reservation.
- Overwrite menghitung delta terhadap size object lama.
- Reconciliation memperbaiki drift terkontrol.

## 11. Placement policy

PlacementTarget fields:

- bucketConnectionId.
- priorityTier.
- weight.
- configuredCapacityBytes.
- thresholdPercent.
- enabled.

Eligibility:

- Target enabled.
- Provider dan bucket healthy atau degraded sesuai policy.
- Usage di bawah threshold.
- Object size dapat ditampung.

- Required capability tersedia.

Selection:

1. Ambil eligible target pada priority tier terendah.
2. Pilih target dengan weighted deterministic hash atau weighted random yang hasilnya langsung disimpan.
3. Buat UploadSession dengan selected target.
4. Jangan menghitung ulang target setelah upload dimulai.
5. Jika upload belum mengirim data dan target gagal, pilih target berikutnya.
6. Jika multipart sudah berjalan, abort sebelum berpindah target.

Policy change hanya memengaruhi upload baru. Existing object pindah melalui migration job terpisah.

## 12. Object index

ObjectRecord menyimpan:

- Organization.
- Namespace.
- Logical key.
- Active object location ID.
- State.
- Size.
- Content type.
- Checksum.
- Created and modified timestamp.

ObjectLocation menyimpan:

- Object record ID.
- Bucket connection dan provider connection.
- Physical key.
- State: CANDIDATE, ACTIVE, RETAINED, atau DELETED.
- ETag dan provider version ID bila tersedia.
- Provider Last-Modified.
- Storage class dan encryption metadata yang portable.
- Verification timestamp.

Semua read menggunakan activeObjectLocationId pada ObjectRecord. Candidate location tidak boleh melayani read sebelum verification dan atomic cutover selesai. Untuk MVP hanya ada satu ACTIVE location; CANDIDATE dan RETAINED dipakai sementara selama migrasi dan bukan replication permanen.

## 13. Cross-provider migration

Scope MVP:

- Memigrasikan current active object, bukan seluruh historical version.

- Mempertahankan logical key, namespace, size, content type, checksum, dan logical created timestamp.
- Memetakan custom metadata, tags, storage class, dan encryption berdasarkan target capability.
- Menjalankan transfer melalui worker dengan bounded memory dan multipart upload.
- Menampilkan preflight, progress object/bytes, kegagalan, dan retained source cleanup status.

MigrationRun memiliki source, destination, namespace atau prefix scope, status, counters, requestedBy, dan retention policy. MigrationItem melacak source fingerprint, candidate location, attempt count, bytes copied, checksum, status, dan error code yang disanitasi. Source fingerprint minimal menggabungkan active location ID, size, checksum, serta provider version ID atau ETag dan Last-Modified bila tersedia.

#### State minimum:

```
PENDING
-> COPYING
-> VERIFYING
-> READY_TO_SWITCH
-> SWITCHED
-> SOURCE_DELETE_PENDING
-> COMPLETED
```

#### Failure dan control states:

```
PAUSED
RETRY_PENDING
FAILED
CANCELED
RECONCILE_REQUIRED
```

#### Rules:

- Cross-provider CopyObject tidak diasumsikan tersedia.
- Migration worker melakukan streaming source GET atau range read ke target multipart upload.
- Size dan checksum independen seperti SHA-256 diverifikasi sebelum cutover; ETag bukan checksum universal. Jika tidak ada trusted source checksum atau target checksum API, worker wajib melakukan target read-back verification sebelum cutover.
- Cutover mengunci ObjectRecord, memeriksa source fingerprint, mengaktifkan candidate ObjectLocation, dan menulis outbox serta audit event dalam satu transaksi.
- Overwrite yang terjadi selama copy membuat source fingerprint stale dan item harus diulang terhadap active version terbaru.
- Delete yang terjadi selama copy membuat tombstone atau terminal state sehingga worker tidak menghidupkan kembali object.
- Write baru mengikuti target migration policy; read selalu mengikuti activeObjectLocationId per object.
- Logical usedBytes tidak bertambah selama temporary double storage. Physical migration bytes dicatat terpisah untuk capacity dan cost visibility.
- Source dipertahankan selama configurable retention window dan sampai presigned URL lama kedaluwarsa.
- Cancel hanya aman sebelum cutover dan harus membersihkan candidate object secara idempotent. Setelah cutover, rollback menggunakan retained source atau reverse migration yang diaudit.
- Semua job idempotent dan dapat dilanjutkan setelah worker restart.

## 14. Audit log

Append-only events:

- Login dan session revocation.
- Organization/member/role changes.
- Provider create/update/delete/test.
- Credential create/rotate/revoke.
- Bucket import.
- Namespace dan placement changes.
- Migration create, pause, resume, cancel, retry, cutover, rollback, dan source cleanup.
- Object delete.
- Quota changes.
- Dedicated gateway assignment.
- Break-glass access.

Audit fields:

- organizationId nullable untuk global action.
- actorType dan actorId.
- action.
- entityType dan entityId.
- Sanitized metadata.
- IP.
- User agent.
- Request ID.
- Created timestamp.

## Data Model Draft

### Identity dan tenancy

| Model          | Field inti                                                 |
|----------------|------------------------------------------------------------|
| User           | id, email, passwordHash, status, createdAt                 |
| Organization   | id, name, slug, status, planId                             |
| Membership     | id, userId, organizationId, status                         |
| Role           | id, organizationId nullable, name, scope                   |
| Permission     | id, key, description                                       |
| RolePermission | roleId, permissionId                                       |
| MemberRole     | membershipId, roleId                                       |
| RefreshToken   | id, userId, tokenHash, expiresAt, revokedAt                |
| ActivityLog    | actor, action, entity, organizationId, metadata, requestId |

## Provider dan namespace

| Model              | Field inti                                                      |
|--------------------|-----------------------------------------------------------------|
| ProviderConnection | id, organizationId, type, endpoint, region, status              |
| ProviderCredential | providerConnectionId, encryptedPayload, keyVersion              |
| ProviderCapability | providerConnectionId, capability, supported, testedAt           |
| BucketConnection   | id, organizationId, providerConnectionId, bucketName, status    |
| VirtualNamespace   | id, organizationId, name, slug, quotaBytes, status              |
| PlacementPolicy    | id, namespaceId, strategy, status                               |
| PlacementTarget    | policyId, bucketConnectionId, tier, weight, capacity, threshold |

## Access dan data

| Model             | Field inti                                                                                                |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| FolderGrant       | namespaceId, prefix, principalType, principalId, actions                                                  |
| ApiCredential     | organizationId, namespaceId, keyId, secretHash, status, expiresAt                                         |
| ObjectRecord      | namespaceId, logicalKey, activeObjectLocationId, state, size, checksum                                    |
| ObjectLocation    | objectRecordId, bucketConnectionId, physicalKey, state, etag, providerVersionId, verifiedAt               |
| UploadSession     | namespaceId, key, targetId, reservedBytes, state, expiresAt                                               |
| MultipartPart     | uploadSessionId, partNumber, etag, size                                                                   |
| UsageCounter      | organizationId, namespaceId, usedBytes, reservedBytes, objectCount                                        |
| UsageEvent        | organizationId, credentialId, operation, bytes, occurredAt                                                |
| OutboxEvent       | aggregateType, aggregateId, eventType, payload, state                                                     |
| ReconciliationRun | targetId, state, scannedCount, driftCount                                                                 |
| MigrationRun      | organizationId, namespaceId, sourceTargetId, destinationTargetId, scope, state, retentionUntil            |
| MigrationItem     | migrationRunId, objectRecordId, sourceLocationId, candidateLocationId, sourceFingerprint, state, attempts |

## Index dan invariant

- Unique Organization.slug.
- Unique VirtualNamespace pada organizationId dan slug.
- Unique ObjectRecord pada namespaceId dan logicalKey untuk non-versioned MVP.
- Unique ApiCredential.keyId.
- Unique UploadSession pada namespaceId dan idempotencyKey.
- Unique MigrationItem pada migrationRunId dan objectRecordId.
- Semua tenant-scoped repository method mewajibkan organizationId.
- activeObjectLocationId tidak boleh null untuk AVAILABLE object.
- Maksimal satu ACTIVE ObjectLocation untuk setiap ObjectRecord.
- CANDIDATE ObjectLocation tidak boleh menjadi activeObjectLocationId sebelum verification berhasil.
- Atomic cutover harus mengubah ACTIVE menjadi RETAINED dan CANDIDATE menjadi ACTIVE dalam satu transaction.
- AVAILABLE object size harus sama dengan committed usage delta.
- Temporary migration copy tidak menambah logical usedBytes.

- Provider secret tidak boleh masuk ActivityLog atau UsageEvent.

## API Contract

### Authentication

| Method | Path             | Guard                   |
|--------|------------------|-------------------------|
| POST   | /v1/auth/login   | Public dan rate-limited |
| POST   | /v1/auth/refresh | Refresh cookie dan CSRF |
| POST   | /v1/auth/logout  | Authenticated           |
| GET    | /v1/auth/session | Authenticated           |

### Organization admin

| Method | Path                          | Permission           |
|--------|-------------------------------|----------------------|
| GET    | /v1/organizations/:id         | organizations:read   |
| PATCH  | /v1/organizations/:id         | organizations:update |
| GET    | /v1/organizations/:id/members | members:read         |
| POST   | /v1/organizations/:id/members | members:create       |
| PATCH  | /v1/members/:id               | members:update       |
| POST   | /v1/members/:id/roles         | roles:assign         |

### Provider dan bucket

| Method | Path                      | Permission       |
|--------|---------------------------|------------------|
| GET    | /v1/providers             | providers:read   |
| POST   | /v1/providers             | providers:create |
| GET    | /v1/providers/:id         | providers:read   |
| PATCH  | /v1/providers/:id         | providers:update |
| DELETE | /v1/providers/:id         | providers:delete |
| POST   | /v1/providers/:id/test    | providers:test   |
| GET    | /v1/providers/:id/buckets | buckets:read     |
| POST   | /v1/buckets/import        | buckets:import   |

### Namespace dan policy

| Method | Path                                | Permission                |
|--------|-------------------------------------|---------------------------|
| GET    | /v1/namespaces                      | namespaces:read           |
| POST   | /v1/namespaces                      | namespaces:create         |
| PATCH  | /v1/namespaces/:id                  | namespaces:update         |
| DELETE | /v1/namespaces/:id                  | namespaces:delete         |
| POST   | /v1/namespaces/:id/targets          | placement_policies:manage |
| PUT    | /v1/namespaces/:id/placement-policy | placement_policies:manage |
| GET    | /v1/folder-grants                   | folder_grants:read        |
| POST   | /v1/folder-grants                   | folder_grants:manage      |
| DELETE | /v1/folder-grants/:id               | folder_grants:manage      |

## API credential

| Method | Path                           | Permission             |
|--------|--------------------------------|------------------------|
| GET    | /v1/api-credentials            | api_credentials:read   |
| POST   | /v1/api-credentials            | api_credentials:create |
| POST   | /v1/api-credentials/:id/rotate | api_credentials:rotate |
| POST   | /v1/api-credentials/:id/revoke | api_credentials:revoke |

## Cross-provider migration

| Method | Path                            | Permission         |
|--------|---------------------------------|--------------------|
| GET    | /v1/migrations                  | migrations:read    |
| POST   | /v1/migrations                  | migrations:create  |
| GET    | /v1/migrations/:id              | migrations:read    |
| POST   | /v1/migrations/:id/pause        | migrations:control |
| POST   | /v1/migrations/:id/resume       | migrations:control |
| POST   | /v1/migrations/:id/cancel       | migrations:control |
| POST   | /v1/migrations/:id/retry-failed | migrations:control |

## Storage client

| Method | Path                                        | Required object action |
|--------|---------------------------------------------|------------------------|
| GET    | /storage/v1/:namespace/objects              | list                   |
| HEAD   | /storage/v1/:namespace/objects/:key         | read                   |
| GET    | /storage/v1/:namespace/objects/:key         | read                   |
| DELETE | /storage/v1/:namespace/objects/:key         | delete                 |
| POST   | /storage/v1/:namespace/uploads              | write                  |
| POST   | /storage/v1/:namespace/uploads/:id/complete | write                  |
| DELETE | /storage/v1/:namespace/uploads/:id          | write                  |

### Upload initiation request:

```
{
  "key": "client-a/invoices/2026-07.pdf",
  "size": 2457600,
  "contentType": "application/pdf",
  "checksum": "sha256:...",
  "idempotencyKey": "upload-123"
}
```

### Direct transfer response:

```
{
  "uploadId": "upl_123",
  "transferMode": "direct",
  "method": "PUT",
  "url": "temporary-presigned-url",
  "headers": {
    "content-type": "application/pdf"
  },
  "expiresAt": "2026-07-31T12:00:00Z"
}
```

### Proxy transfer response:

```
{
  "uploadId": "upl_123",
  "transferMode": "proxy",
  "url": "/storage/v1/uploads/upl_123/content",
}
```

```
"expiresAt": "2026-07-31T12:00:00Z"  
}
```

Every mutation menerima atau menghasilkan requestId. Upload initiation wajib memakai idempotency key.

## Core Workflows

### Register provider

1. Organization admin mengirim provider configuration.
2. API memvalidasi schema dan authorization.
3. Endpoint melewati SSRF validation.
4. Secret dienkripsi.
5. ProviderConnection dibuat sebagai PENDING.
6. Connection test dijalankan.
7. Capability result disimpan.
8. Status menjadi HEALTHY, DEGRADED, atau UNHEALTHY.
9. Audit event dibuat.

### Initiate upload

1. Authenticate API credential.
2. Resolve organization dan namespace.
3. Validate logical key.
4. Evaluate prefix write grant.
5. Enforce maximum object size.
6. Reserve quota transactionally.
7. Select placement target.
8. Create UploadSession.
9. Generate presigned request atau proxy URL.
10. Return short-lived upload contract.

### Complete upload

1. Validate UploadSession ownership dan state.
2. HEAD physical object.
3. Verify size dan checksum bila tersedia.
4. Transactionally create/update ObjectRecord.
5. Convert reservedBytes menjadi usedBytes.
6. Mark session complete.
7. Emit UsageEvent dan audit metadata.

### Read/download

1. Authenticate credential.



2. Evaluate prefix read grant.
3. Resolve ObjectRecord.
4. Check AVAILABLE state.
5. Return proxied stream atau presigned GET.
6. Meter request dan egress sesuai transfer mode.

## Delete

1. Authenticate dan authorize delete.
2. Mark ObjectRecord DELETING.
3. Enqueue provider deletion.
4. Delete physical object.
5. Mark DELETED atau remove record sesuai retention policy.
6. Decrement usage.
7. Audit deletion.

Large recursive delete selalu asynchronous.

## Reconciliation

1. Scan object records dan physical provider secara paginated.
2. Compare expected versus actual metadata.
3. Classify missing, unknown, size mismatch, dan failed delete.
4. Auto-fix hanya safe state.
5. Escalate destructive conflict untuk manual review.
6. Store report dan metrics.

## Migrate current objects

1. Authorize migration action dan validate organization ownership untuk source dan destination.
2. Jalankan target health, credential, capability, capacity, quota, dan scope preflight.
3. Arahkan write baru pada scope migrasi ke destination placement target.
4. Buat MigrationRun serta MigrationItem idempotent dari ObjectRecord yang sesuai scope.
5. Worker membaca source location dan source fingerprint aktif.
6. Stream object ke candidate ObjectLocation pada destination dengan bounded memory.
7. Salin metadata portable dan selesaikan multipart upload bila diperlukan.
8. HEAD target dan verifikasi size serta checksum independen; lakukan target read-back verification bila target tidak memberikan trusted checksum.
9. Lock ObjectRecord dan pastikan source location, checksum, dan modified version belum berubah.
10. Jika source berubah, tandai RETRY\_PENDING dan ulangi dari active version terbaru.
11. Jika valid, atomic switch activeObjectLocationId dan ubah source menjadi RETAINED.
12. Emit outbox, usage, metrics, dan audit metadata tanpa secret.
13. Setelah retention window dan presigned URL expiry, jalankan reconciliation lalu source cleanup.

14. Mark COMPLETED hanya setelah seluruh item terminal dan cleanup policy terpenuhi.

## Admin Panel

### Organization screens

- Overview.
- Members.
- Roles.
- Permissions.
- Providers.
- Provider health dan capabilities.
- Buckets.
- Virtual namespaces.
- Placement policy builder.
- File explorer.
- Folder grants.
- API credentials.
- Upload sessions.
- Migrations: preflight, progress, pause/resume, retry, cancel sebelum cutover, dan cleanup status.
- Usage dan quota.
- Activity logs.

### Platform screens

- Organizations.
- Plans dan feature entitlements.
- Aggregate usage.
- Gateway pools.
- Dedicated gateway assignments.
- Provider compatibility.
- Background jobs.
- Aggregate migration health tanpa default content access.
- Reconciliation reports.
- Audit logs.
- Break-glass access workflow.

Every screen applies:

- Server-side route guard.
- Client action visibility.
- Backend endpoint guard.
- Organization scoping.

# Gateway Deployment Plan

## Standard

- Shared web.
- Shared control API.
- Shared gateway pool.
- Shared PostgreSQL dan Redis.
- App-level per-tenant limits.
- OPNsense aggregate gateway cap.

## Business

- Shared web dan control API.
- Business gateway pool dengan stable VIP.
- App-level per-tenant limits.
- OPNsense pipe per business pool.

## Enterprise

- Shared web dan control API.
- Dedicated gateway deployment.
- Stable private IP atau VIP.
- Tenant hostname.
- OPNsense pipe per tenant.
- Optional dedicated worker concurrency.

## Regulated atau on-premise

- Dedicated deployment dapat mencakup web, API, worker, database, dan Redis.
- Memerlukan pricing, upgrade, backup, dan support model terpisah.

# Roadmap

## Phase 0 - Product dan compatibility spike

Deliverables:

- Lock direct versus proxy product behavior.
- Provider contract test harness.
- AWS, R2, B2, dan MinIO test result.
- Provider capability matrix.
- Threat and failure scenario list.
- Initial OpenAPI draft.

Exit criteria:

- PUT, GET, HEAD, DELETE, list, multipart, dan presigned behavior diketahui.
- Unsupported provider differences terdokumentasi.

- Transfer mode MVP disetujui.

Indicative duration: 1-2 minggu.

## Phase 1 - SaaS foundation

Deliverables:

- Monorepo scaffold.
- Separate web, API, dan worker runtime.
- PostgreSQL, Prisma, dan Redis integration.
- Organization, membership, auth, refresh rotation.
- Action-level RBAC.
- Global dan organization admin baseline.
- Activity log.

Exit criteria:

- Tenant isolation integration tests lulus.
- Role assignment dan permission guards bekerja end-to-end.
- Audit events tercatat tanpa secret.

Indicative duration: 2-3 minggu.

## Phase 2 - Provider dan single-target namespace

Deliverables:

- Provider CRUD.
- Secret encryption.
- SSRF controls.
- Connection test dan capability probing.
- Bucket import.
- Namespace dengan satu target.
- Basic file explorer.

Exit criteria:

- Organization dapat menghubungkan provider sandbox.
- Object operations bekerja pada single target.
- Provider secret tidak muncul di response atau log.

Indicative duration: 3-4 minggu.

## Phase 3 - Client API, grants, dan quota

Deliverables:

- API credential lifecycle.
- Prefix grants.
- Custom storage API.
- UploadSession.

- Quota reservation.
- Direct dan optional proxy transfer.
- Usage events.

Exit criteria:

- Client hanya menerima product API credential.
- Cross-prefix dan cross-tenant access tests gagal dengan benar.
- Concurrent quota test tidak melampaui quota.

Indicative duration: 2-3 minggu.

## **Phase 4 - Multi-provider placement**

Deliverables:

- Multiple placement targets.
- Priority, weight, threshold, dan health eligibility.
- Provider failover sebelum upload.
- ObjectLocation dan migration state model.
- Cross-provider current-object migration worker.
- Multipart streaming, independent checksum verification, dan atomic cutover.
- Minimal migration admin workflow dan API.
- Retained source, rollback window, dan delayed cleanup.
- Reconciliation.
- Orphan cleanup.
- Placement metrics.

Exit criteria:

- Placement distribution sesuai toleransi test.
- Threshold dan tier fallback bekerja.
- Existing object tetap dibaca dari recorded target setelah policy berubah.
- Verified object dapat dipindahkan antar-provider tanpa logical key berubah atau read downtime besar.
- Worker restart, stale source, dan partial failure dapat dilanjutkan secara idempotent.
- Source tidak dihapus sebelum retention serta reconciliation berhasil.

Indicative duration: 3-5 minggu.

## **Phase 5 - Production beta hardening**

Deliverables:

- Rate limiting.
- Gateway backpressure.
- OPNsense shared pool integration.
- Metrics, tracing, dashboards, alerts.

- Database backup/restore validation.
- Load and soak tests.
- Provider outage tests.
- Billing-ready usage export.

Exit criteria:

- Target concurrency dan file size tercapai.
- Gateway tidak buffer full object.
- Recovery procedure diuji.
- Error rate dan SLO beta ditentukan.

Indicative duration: 2-4 minggu.

## Phase 6 - Enterprise gateway

Deliverables:

- Dedicated tenant gateway deployment.
- Stable tenant VIP dan hostname.
- OPNsense per-tenant pipe.
- Tenant-scoped deployment configuration.
- Capacity assignment admin workflow.

Exit criteria:

- Dedicated tenant traffic tidak masuk shared pool.
- Network cap dan app entitlement sama-sama efektif.
- Dedicated gateway tetap menolak credential tenant lain.

## Phase 7 - S3-compatible facade

Deliverables:

- Signature V4 validation.
- S3 XML response/error handling.
- Path-style endpoint.
- ListBuckets dan ListObjectsV2.
- PutObject, GetObject, HeadObject, DeleteObject.
- Multipart create, upload, complete, dan abort.
- Compatibility testing dengan AWS CLI, SDK, rclone, dan Cyberduck.

Exit criteria:

- Supported operation matrix dipublikasikan.
- Unsupported operations memberikan deterministic S3-style errors.
- Credentials tidak dapat keluar dari assigned virtual namespace.

Phase ini tidak termasuk full AWS feature parity.

# Validation Gates

## Security

- Cross-tenant access suite.
- API credential rotation/revocation.
- Provider secret redaction.
- SSRF and DNS rebinding tests.
- Path normalization and prefix bypass tests.
- CSRF and CORS tests.
- Rate-limit abuse tests.
- Cross-tenant migration source/destination rejection.
- Migration permission dan sanitized failure-detail tests.

## Correctness

- Quota concurrency.
- Idempotent upload initiation.
- Multipart retry and abort.
- Overwrite usage delta.
- Delete retry.
- Placement stability.
- Reconciliation drift detection.
- Migration checksum verification dan corruption rejection.
- Atomic location cutover.
- Concurrent overwrite/delete selama migration.
- Idempotent migration retry dan duplicate-job handling.

## Performance

- Large object stream memory profile.
- Concurrent upload/download.
- Provider latency.
- Database query count.
- Queue backpressure.
- Large-object migration bounded-memory profile.
- Per-provider migration concurrency dan bandwidth limits.
- Redis limiter load.
- OPNsense shaping verification.

## Operations

- Database backup and restore.

- Key rotation.
- Provider outage.
- Redis unavailable.
- Worker restart.
- Migration pause/resume, crash recovery, retained-source rollback, dan delayed cleanup.
- Gateway rolling deployment.
- Reconciliation recovery.

## MVP Acceptance Criteria

- Dua organization tidak dapat melihat atau mengakses data satu sama lain.
- Organization admin dapat menambahkan dan menguji provider.
- Organization dapat mengimpor bucket.
- Namespace dapat memakai minimal dua physical bucket.
- File explorer dapat list, upload, download, dan delete.
- API credential dapat dibatasi ke sebuah prefix.
- Revoked credential langsung ditolak.
- Upload melebihi quota ditolak sebelum data dipindahkan.
- Placement priority dan weighted strategy bekerja.
- Provider threshold memindahkan upload baru ke target berikutnya.
- Provider outage sebelum upload dapat memicu failover.
- Existing object selalu menggunakan recorded physical location.
- Organization admin dapat memigrasikan current active object antar-provider tanpa mengubah logical key.
- Read tetap mengikuti lokasi aktif selama copy dan beralih hanya setelah checksum target terverifikasi.
- Partial failure dan worker restart dapat dilanjutkan tanpa duplicate cutover atau logical usage ganda.
- Source dipertahankan sampai retention window dan reconciliation selesai.
- Audit log mencatat security-relevant actions.
- Gateway tidak menyimpan full file dalam memory atau local disk.
- Shared gateway dapat di-scale horizontal.
- Application limit dan OPNsense aggregate cap dapat diverifikasi.

## Scaffold Plan

Tidak ada scaffold aplikasi yang dibuat sebagai bagian dari dokumen ini. Setelah ada konfirmasi implementasi, urutan scaffold yang disarankan:

1. Inisialisasi root npm workspace.
2. Buat apps/web dengan Next.js App Router dan TypeScript.
3. Buat apps/api dengan Express dan TypeScript.



4. Buat apps/worker.
5. Buat packages/shared dan Zod schemas.
6. Buat packages/db dan Prisma.
7. Tambahkan PostgreSQL, Redis, dan MinIO development services.
8. Buat auth dan tenant isolation vertical slice.
9. Buat provider connection vertical slice.
10. Tambahkan CI untuk lint, type check, tests, dan migrations.

Sebelum implementasi dimulai, konfirmasi:

- Lokasi repository.
- npm sebagai package manager.
- Express sebagai API framework.
- Docker Compose untuk development.
- Provider sandbox credentials yang tersedia.
- Pilihan secret manager/KMS untuk production.

## Estimasi

Untuk tim kecil berpengalaman, beta terkontrol secara kasar berada pada rentang 4-5 bulan. Estimasi ini bukan komitmen dan sangat dipengaruhi oleh:

- Provider compatibility issues.
- Proxy versus presigned transfer.
- Target file size dan concurrency.
- Total migration volume, retention window, dan target throughput.
- Infrastruktur self-hosted.
- Security/compliance requirement.
- Kedalaman billing integration.

S3-compatible facade, replication, historical-version migration, advanced migration orchestration, atau on-premise offering harus diestimasi sebagai workstream terpisah.

## Definition of Done

Sebuah feature dianggap selesai bila:

- Functional requirement terpenuhi.
- Authorization dan organization scoping diuji.
- Input tervalidasi.
- Error mapping tidak membocorkan internal details.
- Audit event ditambahkan bila security-relevant.
- Metrics dan logs tersedia.
- Unit/integration test relevan lulus.
- API contract diperbarui.

- Migration aman dan reviewable.
- Operational failure mode terdokumentasi.

## **Referensi Internal**

- Feedback, pro/kontra, dan risiko: ./FEEDBACK.md
- Teknologi, bandwidth, OPNsense, dan deployment: ./TECHNOLOGY.md